

Experiment 11

Name: P.Hithaishi

Course Code: MLA0305

Regno: 192425196

Slot: B

Policy Gradient Implementation using REINFORCE Algorithm

AIM

To implement the REINFORCE policy gradient algorithm using TensorFlow and Keras for solving a Reinforcement Learning problem.

PROCEDURE

1. Set up the Python environment in Google Colab.
2. Import TensorFlow, Keras, NumPy and the required libraries.
3. Create a suitable Reinforcement Learning environment.
4. Define the state space and action space.
5. Construct a neural network to represent the policy.
6. Initialize the policy network and its optimizer.
7. Allow the agent to interact with the environment and select actions using the policy.
8. Store the rewards and selected actions for each episode.
9. Calculate the returns and update the policy using the REINFORCE algorithm.
10. Repeat the training process and evaluate the learning performance using the obtained rewards.

CODE:

```
# EXPERIMENT 11
# Policy Gradient Implementation using REINFORCE
Algorithm

!pip -q install gymnasium

import gymnasium as gym
import numpy as np
import tensorflow as tf
from tensorflow import keras
```

```

tensorflow.keras import layers
import matplotlib.pyplot as plt

# -----
# 1. CREATE ENVIRONMENT
# -----

env = gym.make("CartPole-v1")

state_size = int(env.observation_space.shape[0])
action_size = int(env.action_space.n)

print("State Size :", state_size)
print("Action Size:", action_size)

# -----
# 2. CREATE POLICY NETWORK
# -----

policy = keras.Sequential([
    layers.Input(shape=(state_size,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(action_size, activation="softmax")
])

optimizer = keras.optimizers.Adam(
    learning_rate=0.001
)

episodes = 50
gamma = 0.99

episode_rewards = []

for episode in range(episodes):

    state, info = env.reset()

    states = []
    actions = []
    rewards = []

```

```

done = False
total_reward = 0

while not done:

    state_array = np.asarray(
state,          dtype=np.float32
    )

    # Get action probabilities
probabilities = policy(
state_array.reshape(1, -1),
training=False
    )[0].numpy()

    # Select action according to policy
action = np.random.choice(
action_size,          p=probabilities
    )

    # Take action
next_state, reward, terminated, truncated, info = env.step(action)

    done = terminated or truncated

    # Store experience
states.append(state_array)
actions.append(action)
rewards.append(reward)

    state = next_state
total_reward += reward

    # -----
# 4. CALCULATE DISCOUNTED RETURNS
    # -----

    returns = []
discounted_sum = 0

    for reward in reversed(rewards):

        discounted_sum = (
reward +

```

```

        gamma * discounted_sum
    )

    returns.insert(
0,
        discounted_sum
    )

    returns = np.asarray(
returns,      dtype=np.float32
    )

    # Normalize returns
    if len(returns) > 1:
        returns = (
            returns
            - np.mean(returns)
            ) / (
            np.std(returns) + 1e-8
            )

    states_tensor = tf.convert_to_tensor(
np.asarray(states),
dtype=tf.float32
    )

    actions_tensor = tf.convert_to_tensor(
actions,      dtype=tf.int32
    )

    returns_tensor = tf.convert_to_tensor(
returns,      dtype=tf.float32
    )

    # -----
# 5. POLICY UPDATE
    # -----

    with tf.GradientTape() as tape:

        action_probabilities = policy(
states_tensor,      training=True
        )

        selected_probabilities = tf.gather(
action_probabilities,

```

```

actions_tensor,          axis=1,
batch_dims=1
    )

    log_probabilities = tf.math.log(
selected_probabilities + 1e-8
    )

    loss = -tf.reduce_mean(
log_probabilities * returns_tensor    )

    gradients = tape.gradient(
loss,
    policy.trainable_variables
    )

    optimizer.apply_gradients(
zip(
    gradients,
    policy.trainable_variables
    )
    )

    episode_rewards.append(
total_reward
    )

    print(          f"Episode {episode +
1:2d}/{episodes} | "          f"Reward:
{int(total_reward):3d}"
    )

# -----
# 6. CLOSE ENVIRONMENT
# -----

env.close()

# -----
# 7. PERFORMANCE GRAPH
# -----

plt.figure(figsize=(10, 5))

```

```

plt.plot(
    episode_rewards,
    marker="o",
    label="REINFORCE Reward"
)

plt.xlabel("Episode") plt.ylabel("Total
Reward") plt.title(
    "REINFORCE Policy Gradient Performance"
)

plt.legend() plt.grid(True, linestyle="--
", alpha=0.6)

plt.tight_layout()
plt.show()

# -----
# 8. RESULT
# -----

print("\n=====") print("REINFORCE
TRAINING COMPLETED!")
print("=====")

print(
    "Best Reward :",
    max(episode_rewards)
)

print(
    "Average Reward :",
    round(np.mean(episode_rewards), 2)
)

```

OUTPUT:

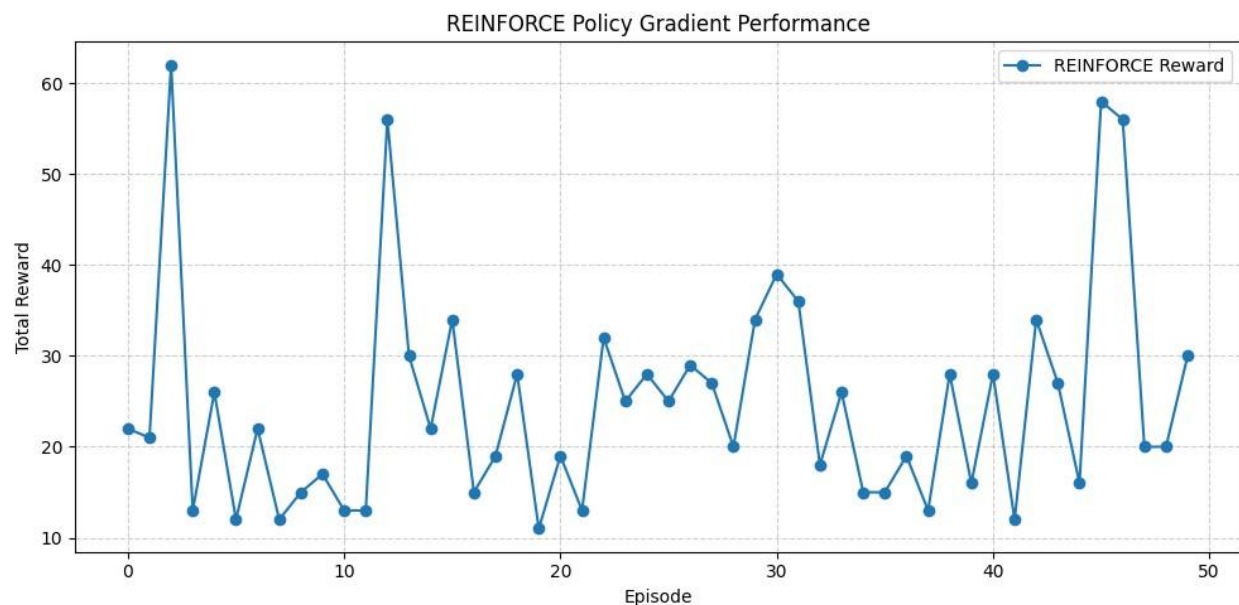
```

State Size : 4
Action Size: 2
Episode 1/50 | Reward: 22
Episode 2/50 | Reward: 21

```

Episode	3/50	Reward:	62
Episode	4/50	Reward:	13
Episode	5/50	Reward:	26
Episode	6/50	Reward:	12
Episode	7/50	Reward:	22
Episode	8/50	Reward:	12
Episode	9/50	Reward:	15
Episode	10/50	Reward:	17
Episode	11/50	Reward:	13
Episode	12/50	Reward:	13
Episode	13/50	Reward:	56
Episode	14/50	Reward:	30
Episode	15/50	Reward:	22
Episode	16/50	Reward:	34
Episode	17/50	Reward:	15
Episode	18/50	Reward:	19
Episode	19/50	Reward:	28
Episode	20/50	Reward:	11
Episode	21/50	Reward:	19
Episode	22/50	Reward:	13
Episode	23/50	Reward:	32
Episode	24/50	Reward:	25
Episode	25/50	Reward:	28
Episode	26/50	Reward:	25
Episode	27/50	Reward:	29
Episode	28/50	Reward:	27
Episode	29/50	Reward:	20
Episode	30/50	Reward:	34
Episode	31/50	Reward:	39
Episode	32/50	Reward:	36
Episode	33/50	Reward:	18
Episode	34/50	Reward:	26
Episode	35/50	Reward:	15
Episode	36/50	Reward:	15
Episode	37/50	Reward:	19
Episode	38/50	Reward:	13
Episode	39/50	Reward:	28
Episode	40/50	Reward:	16

Episode 41/50 | Reward: 28
Episode 42/50 | Reward: 12
Episode 43/50 | Reward: 34
Episode 44/50 | Reward: 27
Episode 45/50 | Reward: 16
Episode 46/50 | Reward: 58 Episode 47/50 |
Reward: 56
Episode 48/50 | Reward: 20
Episode 49/50 | Reward: 20
Episode 50/50 | Reward: 30



===== REINFORCE
TRAINING COMPLETED!

=====

Best Reward : 62.0
Average Reward : 24.82

RESULT

The **REINFORCE policy gradient algorithm** was successfully implemented, and the agent learned a policy through reward-based updates.